UNIT 4

Q1) Comparison Between Finite Automata (FA) and Push Down Automata (PDA)

Aspect	Finite Automata (FA)	Push Down Automata (PDA)
Definition	A finite automaton is a computational model that accepts or rejects strings based on a finite set of states and transitions between them.	A pushdown automaton is a computational model that accepts or rejects strings based on finite states and transitions, but it also has an additional stack for memory.
Memory Structure	FA has no additional memory; it only uses its finite set of states.	PDA has a stack which allows it to store and retrieve information as needed.
States	FA has a finite number of states.	PDA also has a finite number of states, but the stack provides additional memory.
Transition Function	Transitions in FA depend only on the current state and input symbol.	In PDA, transitions depend on the current state, input symbol, and the top symbol of the stack.
Accepting Mechanism	FA accepts a string if it ends in an accepting state.	PDA accepts a string either by reaching an accepting state or by the stack being empty (depending on the type of acceptance).
Power	FA can recognize only regular languages.	PDA can recognize context-free languages, which are more complex than regular languages.
Determinism	FA can be deterministic (DFA) c ↓ n-deterministic (NFA).	PDA can also be deterministic (DPDA) or non-deterministic (NPDA).

Q2) Post Machine

In 1936, Emil Post introduced a simplistic model which is known as Post's Machine. This machine has a capability to accept any language that can be precisely defined by human beings.

This machine is more powerful than finite automata and push down automata. That means it can accept regular, non-regular, context-free and non-context-free languages.

The Post Machine (PM) is a collection of following things -

1. The set of input letters denoted by ∑, along with a special symbol #.
2. The linear storage location in which the input symbols can be placed. Such a location is called STORE or QUEUE. The symbols can be inserted in this QUEUE from one end and deleted from other end.

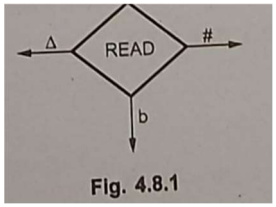

Fig. 4.8.1

3. The READ state which is deterministic in nature.
4. The ADD state used to insert the symbol in queue.
5. The START state, ACCEPT and REJECT states. The ACCEPT state in usually halt state.

The post machine is normally deterministic in nature but sometimes non-deterministic post machine is also used.

Working of a Post Machine:

- The machine begins with a string of symbols written on the tape.
- The machine reads the tape from left to right and applies the first rule it encounters (or rules, if multiple rules apply). This results in the modification of the tape.
- The machine then continues reading the tape and applies more rules, repeating this process until no more applicable rules can be found or until the machine halts.

Q3)Define Push down Automata. Explain different types of PDA. Explain any two applications of PDA. [8]

Definition

The PDA can be defined as a collection of seven components.

1. The finite set of states Q.
2. The input set Σ.
3. Γ is a stack alphabet.
4. q_0 is initial stage, $q_0 \in Q$.
5. Z_0 is a start symbol which is in Γ.
6. Set of final states $F \subseteq Q$.
7. δ is mapping function used for moving from current state to next state.

- The push down automata will have input tape, finite control, and stack.
- The input tape is divided in many cells. At each cell only one input symbol is placed thus certain input string is placed on tape.
- The finite control has some pointer which points to the current symbol which is to be read.
- At the end of input $ or Δ (blank) 'b' (blank) symbol is placed which indicates end of input.
- The stack is such a structure in which you can push and remove the items from one end only.
- In the push down automata, we are using the stack for storing the items temporarily.
- Inserting the symbol onto the stack is called **push operation** and removing a symbol from stack is called **pop operation**.

Types of Pushdown Automata (PDA)

There are two main types of PDAs based on how the transitions are determined:

1. **Deterministic Pushdown Automaton (DPDA)**:
   - In a DPDA, for every state and input symbol (including the stack top symbol), there is at most one possible transition.
   - This means the transition function is deterministic and does not allow ambiguity in terms of which transition to choose.
   - DPDAs are less powerful than non-deterministic PDAs because some context-free languages cannot be recognized by a DPDA.
   - **Example**: A PDA that recognizes the language $L = \{a^n b^n | n \geq 0\}$ is deterministic.
2. **Non-deterministic Pushdown Automaton (NPDA)**:
   - In an NPDA, for a given state, input symbol, and stack top symbol, the machine may have multiple possible transitions.
   - This non-determinism means the PDA can "guess" the correct path to follow, exploring all possible configurations and accepting if one configuration leads to an accepting state or an empty stack.
   - NPDAs are more powerful than DPDAs because they can recognize all context-free languages (CFLs).
   - **Example**: A PDA that recognizes the language $L = \{a^n b^n c^n | n \geq 0\}$ requires non-determinism.

Applications of Pushdown Automata

1. Compiler Design (Parsing) : One of the most important applications of PDAs is in **parsing** in the design of compilers. PDAs are used in syntax analysis to process the grammar of programming languages, which is often context-free.

In parsing, the input string is processed symbol-by-symbol, and the stack is used to store intermediate results. The PDA ensures that the structure of the input string matches the grammar's production rules.

For example, a PDA can be used to check whether a string of parentheses is balanced, which is a typical application of context-free languages.

2. Syntax Checking for Balanced Parentheses and Nested Structures

Another practical use of PDAs is in the validation of **balanced parentheses**, which is essential in many programming languages. A PDA can easily verify whether every opening parenthesis has a corresponding closing parenthesis, and it can also handle other nested structures like brackets, braces, and even HTML/XML tags.

The PDA accepts the string if, after reading the entire string, the stack is empty (indicating all opening symbols have been matched with closing symbols).

For example:

- The string "(a + b) * (c + d)" is valid because every opening parenthesis has a matching closing parenthesis.
- The string "(a + b * (c + d)" is invalid because the parentheses are not balanced.

Q4) Write a note on Instantaneous Description of PDA with an example

- For representing certain problems FA is not sufficient, we need PDA wherein there is an effective use of stack as a memory.
- We can describe this behaviour of PDA by means of **instantaneous description**. It is given by ID.

- The moves of ID's are as shown below.

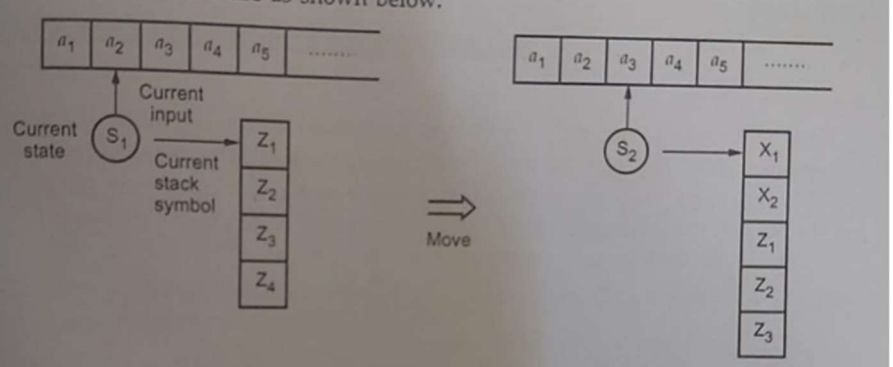

Fig. 4.3.1

- From Fig. 4.3.1, if we are reading the current symbol a_2, at current state S_1 and current stack symbol Z_1 then after a move we will reach to state S_2 and there will be some new symbol on the top of the stack.

be some new symbol on the top of the stack.

- This description can be represented as -

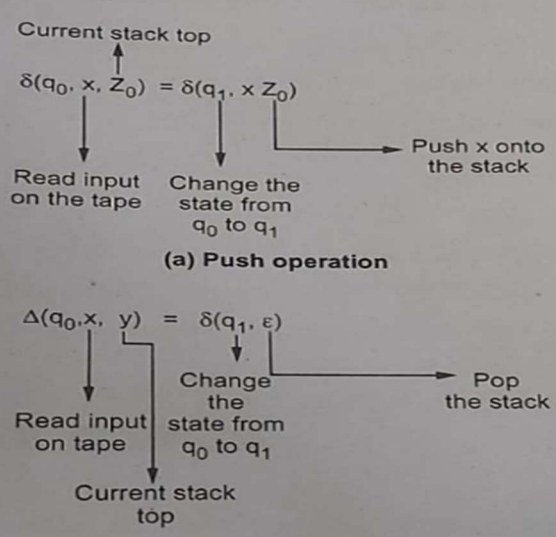

Fig. 4.3.2 Instantaneous description

Q5) Explain the acceptance by PDA [6] i) By final state ii) By empty stackA

Pushdown Automaton (PDA) can accept an input string in two distinct ways:

1. **Acceptance by Final State**
2. **Acceptance by Empty Stack**

These two methods of acceptance define how a PDA determines whether a given string belongs to the language it recognizes.

i) Acceptance by Final State

In the **acceptance by final state** method, the PDA accepts an input string if, after processing the entire input string, it reaches an accepting (final) state. The stack contents are not necessarily important in this case. The primary condition for acceptance is that the machine must be in a final state when the input string is completely read.

Process:

1. The PDA starts in its initial state and begins processing the input string symbol by symbol.
2. The PDA uses its transition function to move between states and possibly push or pop symbols onto/from the stack based on the input symbol.
3. If, after reading the entire input string, the PDA ends in one of its accepting states, then the input string is considered **accepted**.

Example:

Consider a PDA that recognizes the language $L = \{a^n b^n | n \geq 0\}$ using acceptance by final state.
The PDA could have the following configuration:

- **States**: q_0, q_1 (where q_0 is the initial state and q_1 is the accepting state).
- **Stack Alphabet**: $\{Z_0, a\}$.
- **Input Alphabet**: $\{a, b\}$.
- **Transitions**:
  - $q_0, a, Z_0 \rightarrow (q_0, aZ_0)$ (push 'a' onto the stack on reading 'a').
  - $q_0, b, a \rightarrow (q_0, \epsilon)$ (pop 'a' from the stack on reading 'b').
  - $q_0, \epsilon, Z_0 \rightarrow (q_1, Z_0)$ (move to q_1 when the input is empty and the stack contains only Z_0)

For input string $w = \text{"aabb"}$, the PDA will:

- Start in state q_0, push symbols onto the stack when reading 'a's, and pop symbols when reading 'b's.
- At the end of the input, it will reach state q_1, which is the accepting state, and accept the string.

Thus, the PDA **accepts** w by reaching the final state q_1.

ii) Acceptance by Empty Stack

In the **acceptance by empty stack** method, the PDA accepts an input string if, after processing the entire input string, the stack is empty. The final state is not necessarily important here. The primary condition for acceptance is that the stack should be empty after all input symbols are processed.

Process:

1. The PDA starts in its initial state and processes the input string symbol by symbol.

2. During the processing, the PDA pushes symbols onto the stack when reading certain input symbols and pops symbols from the stack when reading others.

3. If, after reading the entire input string, the stack is empty, the PDA **accepts** the string, regardless of the state it ends in.

Example:

Consider a PDA that recognizes the language $L = \{a^n b^n | n \geq 0\}$ using acceptance by empty stack.
The PDA configuration could be:

- **States**: q_0 (initial state), q_1 (accepting state, though not necessary for acceptance).
- **Stack Alphabet**: $\{Z_0, a\}$.
- **Input Alphabet**: $\{a, b\}$.
- **Transitions**:
  - $q_0, a, Z_0 \rightarrow (q_0, aZ_0)$ (push 'a' onto the stack on reading 'a').
  - $q_0, b, a \rightarrow (q_0, \epsilon)$ (pop 'a' from the stack on reading 'b').
  - $q_0, \epsilon, Z_0 \rightarrow (q_0, Z_0)$ (no transition required, but keeps the stack unchanged).

For input string $w = \text{"aabb"}$, the PDA will:

- Start in state q_0, push 'a' onto the stack for each 'a', and pop 'a' for each 'b'.
- After reading the entire string, the stack will be empty.

Thus, the PDA **accepts** w by emptying the stack, even if it does not reach any final state.

Comparison of the Two Methods

1. **Acceptance by Final State**:
   - The machine accepts the string if it ends in an accepting state after consuming the entire input.
   - The stack contents are not important.
   - It is easier to implement for simple languages where the state alone determines the acceptance condition.
2. **Acceptance by Empty Stack**:
   - The machine accepts the string if the stack is empty after consuming the entire input, regardless of the state.
   - It is often used in more complex languages, especially when the stack is critical for determining the acceptance condition.
   - This method provides more flexibility but may require more careful management of the stack during the computation.